

LAB 2: BACKEND VỚI NODEJS & EXPRESS

Nhóm 2 | Chủ đề: Xây dựng RESTful API Quản lý Đơn hàng (CRUD)

I. MỤC TIÊU BÀI HỌC

- Hiểu cách khởi tạo và cấu trúc project Express theo mô hình MVC đơn giản.
- Kết nối và thao tác với cơ sở dữ liệu MongoDB Atlas thông qua Mongoose.
- Thành thạo các phương thức HTTP: GET, POST, PUT, DELETE cho resource Đơn hàng.
- Sử dụng Postman để kiểm thử toàn bộ luồng CRUD của API.

II. CHUẨN BỊ (Prerequisites)

- Node.js đã cài đặt (phiên bản ≥ 18).
- Postman hoặc VS Code Extension "Thunder Client".
- Tài khoản MongoDB Atlas (đã tạo Cluster và lấy Connection String).
- Trình soạn thảo VS Code.

III. NỘI DUNG THỰC HÀNH TRÊN LỚP

Bước 1: Khởi tạo Project và Cài đặt thư viện

Tạo thư mục dự án và cài đặt các thư viện cần thiết.

```
mkdir order-management-api
cd order-management-api
npm init -y

# Cài đặt các thư viện
npm install express mongoose dotenv cors

# Cài đặt nodemon (chế độ dev)
npm install --save-dev nodemon
```

Cấu hình file package.json: Thêm dòng sau vào phần "scripts":

```
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js"
}
```

Bước 2: Cấu trúc thư mục dự án

Tổ chức code theo cấu trúc rõ ràng từ đầu:

```
order-management-api/
├── models/
│   └── Order.js
├── routes/
│   └── orderRoutes.js
├── .env
├── .gitignore
└── server.js
```

Bước 3: Cấu hình biến môi trường và Kết nối MongoDB

Tạo file **.env**:

```
PORT=5000
MONGO_URI=mongodb+srv://<username>:<password>@cluster0.xxx.mongodb.net/OrderDB?retryWrites=true&w=majority
```

Tạo file **server.js**:

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const app = express();

// Middleware
app.use(cors());
app.use(express.json()); // Cho phép server đọc JSON từ request body

// Kết nối MongoDB
mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('✅ MongoDB Connected!'))
  .catch(err => console.error('❌ Connection error:', err));

// Route kiểm tra server
app.get('/', (req, res) => {
  res.send('API Quản lý Đơn hàng đang hoạt động...');
});

// Khởi chạy server
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`🚀 Server is running on port ${PORT}`);
});
```

Bước 4: Định nghĩa Model Đơn hàng

Tạo file **models/Order.js**:

```
const mongoose = require('mongoose');

const OrderSchema = new mongoose.Schema({
  customerName: { type: String, required: true },
  customerEmail: { type: String, required: true },
  items: [
    {
      productName: { type: String, required: true },
      quantity: { type: Number, required: true, min: 1 },
      unitPrice: { type: Number, required: true }
    }
  ],
  totalAmount: { type: Number, required: true },
  status: {
    type: String,
    enum: ['pending', 'confirmed', 'shipped', 'delivered', 'cancelled'],
    default: 'pending'
  },
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Order', OrderSchema);
```

Bước 5: Viết các Route CRUD cho Đơn hàng

Tạo file **routes/orderRoutes.js**:

```
const express = require('express');
const router = express.Router();
const Order = require('../models/Order');

// 1. Lấy toàn bộ đơn hàng (GET /api/orders)
router.get('/', async (req, res) => {
  try {
    const orders = await Order.find().sort({ createdAt: -1 });
    res.json(orders);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// 2. Lấy đơn hàng theo ID (GET /api/orders/:id)
router.get('/:id', async (req, res) => {
  try {
    const order = await Order.findById(req.params.id);
    if (!order) return res.status(404).json({ message: 'Không tìm thấy đơn hàng' });
    res.json(order);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
```

```
// 3. Tạo đơn hàng mới (POST /api/orders)
router.post('/', async (req, res) => {
  const order = new Order({
    customerName: req.body.customerName,
    customerEmail: req.body.customerEmail,
    items: req.body.items,
    totalAmount: req.body.totalAmount
  });
  try {
    const newOrder = await order.save();
    res.status(201).json(newOrder);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

// 4. Cập nhật trạng thái đơn hàng (PUT /api/orders/:id)
router.put('/:id', async (req, res) => {
  try {
    const updatedOrder = await Order.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );
    if (!updatedOrder) return res.status(404).json({ message: 'Không tìm thấy đơn hàng' });
    res.json(updatedOrder);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

// 5. Xóa đơn hàng (DELETE /api/orders/:id)
router.delete('/:id', async (req, res) => {
  try {
    const deleted = await Order.findByIdAndDelete(req.params.id);
    if (!deleted) return res.status(404).json({ message: 'Không tìm thấy đơn hàng' });
    res.json({ message: 'Đã xóa đơn hàng thành công!' });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

module.exports = router;
```

Nhúng route vào server.js (thêm sau phần Middlewares):

```
const orderRoutes = require('./routes/orderRoutes');
app.use('/api/orders', orderRoutes);
```

IV. KIỂM THỬ VỚI POSTMAN

Chạy server: `npm run dev`

Method	Endpoint & Mô tả
POST	<code>http://localhost:5000/api/orders</code> — Tạo đơn hàng mới
GET	<code>http://localhost:5000/api/orders</code> — Lấy toàn bộ đơn hàng
GET /:id	<code>http://localhost:5000/api/orders/<_id></code> — Lấy đơn hàng theo ID
PUT /:id	<code>http://localhost:5000/api/orders/<_id></code> — Cập nhật đơn hàng
DELETE /:id	<code>http://localhost:5000/api/orders/<_id></code> — Xóa đơn hàng

Ví dụ JSON Body cho POST:

```
{
  "customerName": "Nguyen Van A",
  "customerEmail": "vana@email.com",
  "items": [
    { "productName": "Laptop Dell XPS", "quantity": 1, "unitPrice": 25000000 },
    { "productName": "Chuot Logitech", "quantity": 2, "unitPrice": 500000 }
  ],
  "totalAmount": 26000000
}
```

Ví dụ JSON Body cho PUT (cập nhật trạng thái):

```
{
  "status": "confirmed"
}
```

V. PHẦN MỞ RỘNG (Gợi ý Lab hay hơn)

Giảng viên nên hướng dẫn thêm các kỹ thuật sau để nâng chất lượng bài lab:

- Validation nâng cao: Kiểm tra `totalAmount` phải bằng tổng (`quantity × unitPrice`) của tất cả items, nếu sai thì trả lỗi 400.
- Response chuẩn hóa: Bọc toàn bộ response trong object `{ success: true, data: ..., message: "..." }`.
- Middleware Logging: Cài morgan để in log request lên terminal trực quan hơn.

VI. BÀI TẬP TỰ LÀM (Challenge)

Yêu cầu 1: Lọc theo trạng thái đơn hàng

Viết API: `GET /api/orders?status=pending` để lọc đơn hàng theo trạng thái.

Gợi ý: Sử dụng `req.query.status` và `Order.find({ status: req.query.status })`.

Yêu cầu 2: Tìm kiếm theo tên khách hàng

Viết API: `GET /api/orders/search?name=...` để tìm kiếm theo tên khách hàng.

Gợi ý: Sử dụng Mongoose Regex: `{ customerName: { $regex: req.query.name, $options: 'i' } }`.

Yêu cầu 3: Sắp xếp theo tổng tiền

Viết API: `GET /api/orders?sort=asc` hoặc `desc` để sắp xếp theo `totalAmount`.

Gợi ý: Sử dụng `.sort({ totalAmount: sortOrder === 'asc' ? 1 : -1 })`.

VII. HƯỚNG DẪN DEPLOY (Nhanh)

- **Database:** Sử dụng MongoDB Atlas (đã thiết lập ở bước 3).
- **Backend:** Đẩy code lên GitHub, sau đó kết nối với Render.com (Free) hoặc Railway.app.
- **Lưu ý quan trọng:** Phải cấu hình Environment Variables (PORT, MONGO_URI) trên giao diện Render/Railway thì app mới chạy được.

💡 **Lời khuyên cho sinh viên:** "Đơn hàng thật ngoài đời có thể sai — nhưng API của bạn không được phép sai! Hãy test kỹ từng endpoint trước khi nộp bài."